Assignment 02 Threads and processes

1 Processes and threads

1. **Explain the difference between a process and a thread.**
   A process is an instance of a program and have a virtual address space, executable and a PID. A thread is sequence of instructions that inherits the address space of an existing process and can be scheduled for execution. Thread runs in the a shared memory space, but processes doesn't. Threads also have less overhead then processes. Both have their own version of a control block; PCB and TCP
2. **Describe a scenario where it is desirable to**
   - **Write a program that uses threads for parallel processing**
   If the case is to send a lot of HTTP POST request at the same time to a endpoint you can schedule threads to run at the same time
   - **Write a program that uses processes for parallel processing**
   If the case is needing to isolate each parallell process's address space, use multiple cores, and have more control of each process. Ex: image processing on many files
3. **Explain why each thread requires a thread control block (TCB).**
   A TCB is a data structure that holds the thread's state and consist of data unique to each thread's execution context, which allow the OS to schedule threads. A TCB controls all the necessary information to track and control the execution of a thread. Two types of information it holds: state and metadata. In short, thread requires TCB otherwise it would be very unsafe to use threads and difficult to keep track of each thread.
4. **What is the difference between cooperative (voluntary) threading and pre-emptive (involuntary) threading? Briefly describe the steps necessary for a context switch for each case.**
   Cooperative threading involves voluntary yielding the CPU so that another thread can be scheduled. Command to use `thread_yield()`. Preemptive threading is the opposite - involuntary giving up the CPU via an interrupt or exception which wakes up the kernel and tell us to switch threads.

Short each case:
When implementing voluntary context switch (kernel or user threads) you need to save the state of current thread by saving registers on thread's kernel stack, switch to new stack and new thread, resume execution which restores registers from new thread and return.

When having a preemptive threading schedule, we need to have an event to trigger the involuntary preemption of the thread, so we have to take the CPU from it and give it to another thread. This is done with a timer og I/O interrupt. We can also use `switch()` at the end of interrupt handler.

2 C program with POSIX threads

```c
#include <stdio.h>
#include <pthread.h>

#define NTHREADS 10

pthread_t threads[NTHREADS];

void *go(void *n)
{
  printf("Hello from thread %ld\n", (long)n);
  pthread_exit(100 + n);
  // REACHED?
}

int main()
{
  long i;
  for (i = 0; i < NTHREADS; i++)
    pthread_create(&threads[i], NULL, go, (void *)i);

  for (i = 0; i < NTHREADS; i++)
  {
    long exitValue;
    pthread_join(threads[i], (void *)&exitValue);
    printf("Thread %ld returned with %ld\n", i, exitValue);
  }

  printf("Main thread done.\n");
  return 0;
}
```

1. **Which part of the code (e.g., the task) is executed when a thread runs? Identify the function and describe briefly what it does.**
   The function `go()` and a function pointer is called as an argument in `pthread_create()` which will be executed in the new thread. It prints a string element and the pointer to the variable n, and terminates the calling thread.
2. **Why does the order of the "Hello from thread X" messages change each time you run the program?**
   `pthread_create()` creates a new thread, initializes and starts the thread to run the given function `go()`. It makes the thread runnable/placed in *ready* state. The order changes because all the function are executed concurrently in threads and the OS picks any scheduled thread in *ready* state and each function have variable runtime.
   Also, `pthread_create()` unlike `pthread_join()` will not wait for the specific thread to terminate and will not block the calling thread until the target thread finishes.
3. **What is the minimum and maximum number of threads that could exist when thread 8 prints "Hello"?**
   Maximum: 10 new threads are started (including thread that prints 8 and the program itself which is a thread).
   Minimum: Only the main thread (program) and thread that prints 8 have started and is in *running* state. The rest has not started yet or finished already.
4. **Explain the use of pthread join function call.**
   This function waits for the specific thread to terminate and blocks the calling thread until the target thread finishes. If that thread has already terminated, then `pthread_join()` returns immediately. Used to synchronize the execution of threads.

Source: I ran `man pthread_join()` in NTNU linux server

5. **What would happen if the function go is changed to behave like this:**

```c
void *go (void *n) {
    printf("Hello from thread %ld\n", (long)n);
    if(n == 5) sleep(2); // Pause thread 5 execution for 2 seconds
    pthread_exit(100 + n);
    // REACHED?
}
```

The thread with value 5 get scheduled and is running. Right before calling `pthread_exit()`, it sleeps for 2 seconds. The main thread pauses at `pthread_join(thread[5])` until the sleep(2) is over. We can observe in the output that is pauses 2 seconds before joining thread 5 and printing "Thread 5 returned with 105....." in order.

6. **When pthread_join() returns for thread X, in what state is thread X?**

When it returns, the target thread is terminated (it no longer exist). It is therefore in the *finished* state. `pthread_join()` block until the target (thread X) is terminated.